DUNE

Indoor Ultra-Wideband Positioning for a Small Rover

A complete technical account — how it works, how to run it, why it was built this way, and what failed along the way

Repository	`https://github.com/AdershM-2/UWB_3`
Date	22 July 2026
Host software	MATLAB R2024b
Radio hardware	DecaWave DW1000 + ESP32

This document is written to be *read*, in order, by someone who has never seen the project. Every idea is explained in plain sentences before any mathematics appears, every equation is translated into words, and every component links to the exact source file that implements it. Green boxes contain commands you can run; red boxes describe mistakes we made so you do not repeat them.

Contents

Part I

The system

1 The big picture

The goal is simple to state: know where a small rover is inside a room, to within a few centimetres, many times per second — and know which way it is pointing.

The tool is ultra-wideband (UWB) radio. Five small radio stations, called **anchors**, are fixed around the room at known positions. One or two radios called **tags** ride on the rover. A tag can measure its distance to an anchor by timing how long a radio message takes to fly there and back. Radio moves at the speed of light — about 30 cm every nanosecond — so to get centimetres the timing must be good to a few tens of *picoseconds*. The DW1000 radio chip is built for exactly this.

In plain words

Think of each distance measurement as a piece of string from an anchor to the rover. One string tells you the rover is somewhere on a circle. Two strings narrow it to two points. Three or more pin it down. Because every string has a little error, the strings never agree perfectly, so we look for the point that keeps *all* of them as close to happy as possible. Everything else in this document — calibration, filtering, two-tag geometry — exists to make the strings more truthful and the disagreement-settling smarter.

The data flows through five stages, each covered by a part of this document:

1. **Ranging** (Part III). The tag performs a timed message exchange with each anchor in turn and reports the distances over USB or WiFi.
2. **Correction** (Part III). Each raw distance is corrected for two known instrument errors: the antenna's internal delay, and a bias that depends on signal strength.
3. **Solving** (Part IV). Corrected distances become a position by weighted least squares.
4. **Filtering** (Part IV). Positions are smoothed over time by a Kalman filter or a moving-horizon estimator, which know that rovers move smoothly.
5. **Rigid-body fusion** (Part V). With two tags bolted to one plate, we estimate the rover's whole pose — centre, heading, speed — in one solve.

Independently of all that, a Kinect camera on the ceiling watches a printed AprilTag marker on the rover. After careful calibration (Part II) the camera is trusted as **ground truth**: it tells us where the rover *really* is, so we can honestly score how well the radio system does.

1.1 Numbers at a glance

2 Hardware and room setup

Radio configuration (set in `UwbConfig.h`): channel 5, data rate 110 kb/s, pulse repetition 64 MHz, long preamble. This is the chip's most accurate mode — slow but precise, the right trade for a 6 Hz system.

Table 1: Headline performance, all measured (not simulated unless stated).

Quantity	Value	Where shown
Single-reading accuracy, parked	94 mm RMSE (19 spots)	§9
Filtered accuracy, parked	60–65 mm	§12
Reference-point accuracy after calib.	4 mm	§6.1
Update rate	5–7 Hz (one tag)	§5.2
Heading from two tags	7.3° RMS vs truth	§15
Rigid-body accuracy (simulation)	14 mm centre, 1.2°	§13.2
Known residual drift, parked	5–7 cm over minutes	§11

Table 2: The physical pieces.

Piece	Count	Details
Anchor	5	DW1000 UWB radio on an ESP32 host board, battery powered, mounted **24 cm** above the floor around a ∼6×3 m area. Firmware: `Anchor.ino`.
Tag 240 (`0xF0`)	1	As above, plus a BNO085 IMU (orientation, gyro, accelerometer). Mounted at the **rear** of the rover plate. Firmware: `TagWrover.ino`.
Tag 241 (`0xF1`)	1	No IMU. Mounted at the **front**. Firmware: `Tag.ino`.
Rover	1	Six-wheel, front-steering (Ackermann) — it cannot slide sideways, a fact we exploit in §12.3. Carries the two tags **52.7 cm** apart and an AprilTag marker.
Kinect v2	1	On the ceiling at 4.066 m, looking down. Ground truth camera.
Host PC	1	Runs MATLAB R2024b. All computation happens here; the radios only measure and report.

The five anchor positions are stored in `anchors.json`. They were not measured with a tape: they were *clicked* in the calibrated ceiling camera (§4.5), which is both faster and more accurate.

3 The repository, and your first live position

Table 3: Repository layout. Names link to the code on GitHub.

Folder	Contents
`matlab/+dune/`	The core library: parsing, solving, filters, transports. Everything callable as `dune.*`.
`matlab/scripts/`	Runnable tools: live displays, calibration, analysis, replay.
`matlab/vision/`	Camera calibration and click-to-world tools.
`matlab/config/`	Anchor layout and range corrections (JSON, human-readable).
`libraries/UwbRtls/`	The firmware library, vendored so the repo is self-contained.
`sketches/`	The per-board Arduino sketches.
`docs/`	The living plan, findings documents, and this report.

Try it yourself

With one tag plugged into USB (it enumerates as a COM port; tag 240 was COM12 here):

```
addpath('matlab', 'matlab/scripts')
live_tag("COM12")
```

A map opens showing the five anchors (green triangles), a grey ghost dot (the raw per-reading solve) and a red dot (the filtered position). The title bar shows position, update rate, how

many anchors answered, and the solver residual. If an anchor stops answering its triangle turns red and the title names it (`MISS: A2`) — the batteries are the usual suspect.
Over WiFi instead of USB (the tags always broadcast; you just have to listen):

```
udp_probe(15)              % first: confirm packets actually arrive
live_tag(transport="udp")
```

Every live session writes two logs to `matlab/logs/`: a JSONL file of parsed, solved records, and a raw byte-for-byte serial/UDP tee. Everything in this project is replayable offline from those logs — a design decision that paid for itself many times over, because every bug could be reproduced without touching hardware.

Part II

Ground truth: the ceiling camera

4 Calibrating the Kinect until it can be trusted

Before believing any radio measurement, we needed an independent instrument that is *more* accurate than the thing being tested. A downward-looking camera can be that instrument, but only after calibration. This section is the story of that calibration, including two wrong answers on the way — kept here deliberately, because the *method* of catching the wrong answers is the useful part.

4.1 The geometry of looking straight down

For a camera at height h looking down, a flat object on the floor appears with a size proportional to f/h, where f is the focal length in pixels. One pixel on the sensor corresponds to h/f metres on the floor. With $h \approx 4\,\mathrm{m}$ and $f \approx 1068\,\mathrm{px}$, one pixel is about 3.8 mm of floor. Sub-centimetre truth is therefore possible — *if* h and f are right. Get h wrong by 5 % and every distance the camera reports is wrong by 5 %.

4.2 Measuring the height by parallax

You cannot hang a tape measure from the ceiling and call it done — the optical centre of the camera is somewhere inside the housing. Instead we measured h with the camera itself, using **parallax**: raise an object by a known amount and watch how far its image shifts.

Consider a point on the floor at radial image distance ρ_1 from the point directly beneath the camera (the *nadir*). Raise it by Δz (we used a box of measured thickness 0.355 m) and it appears further out, at ρ_2, because it is now closer to the camera. Similar triangles give

$$\frac{\rho_2}{\rho_1} = \frac{h}{h - \Delta z} \qquad \Longrightarrow \qquad h = \Delta z \, \frac{\rho_2}{\rho_2 - \rho_1}. \tag{1}$$

In plain words

Hold your thumb at arm's length and raise it towards your eye: it drifts away from the centre of your view. How *fast* it drifts depends on how far away your eye is. The camera's height falls out of the same effect, measured in pixels.

Pitfall — this cost us real time

Our first height estimate, 3.873 m, came from a single sloppy session and was simply wrong. Two careful repeats (box hops in all four quadrants of the image, plumb-bob to transfer the floor point) both said ≈4.21 m — reproducible, so we almost believed it. It was *also* wrong, by about 3 %: uncorrected image distortion biases raw-pixel parallax high. The final method (§4.3) works in floor coordinates instead of raw pixels and gave **4.066 m** with a 7.7 mm fit residual. The lesson: reproducibility is necessary but not sufficient — a systematic bias reproduces beautifully.

4.3 Focal length, and the consistency test that settled everything

The refined procedure (implemented in `calibrateOverheadCamera.m` and friends) solves for height and focal length together, using box hops expressed in metric floor coordinates through a fitted floor homography. It returned $h = 4.066$ m and $f = 1067.6$ px.

The number was accepted only after an *independent* cross-check. The ratio h/f is metres-per-pixel on the floor, and we can measure that directly by laying tape measures in the image (`addScaleBars.m`). The two agreed to 0.2 %:

$$h/f = 4.066/1067.6 = 3.808\,\text{mm/px} \qquad \text{vs} \qquad 3.815\,\text{mm/px from tape.}$$

When two genuinely independent methods agree that closely, the calibration is done.

4.4 Lens distortion: measured, then deliberately ignored

Wide lenses bend straight lines. We photographed stretched strings across the field and measured the bow: **under 1.5 pixels everywhere** — about 6 mm of floor, at the edge of the field, worst case. We then tried fitting a distortion model anyway, and it confidently produced a distortion centre far from the image centre: it was fitting our mouse-click noise, not the lens. The model was discarded and the lens declared straight. Knowing when *not* to model something is calibration skill too.

4.5 Tying the camera to the room, and clicking the anchors in

The camera reports positions in its own frame. To connect it to the room we placed the AprilTag at several positions, clicked its corners, and solved the rigid transform camera→world (`registerWorldFrameClicks.m`). Registration residual: **13.5 mm RMS**, worst point 18.7 mm.

Pitfall — this cost us real time

The printed AprilTag measures 17.78 cm, but the detector behaves as if it were **16.9 cm** — a real property of how the detector defines a tag's border, not a printing error. Until this was anchored against tape measurements, every camera distance carried a hidden 5 % scale error. If you use AprilTags for metrology, calibrate the *effective* size, not the printed one.

With registration done, the camera became a point-measuring machine: click any pixel whose height above the floor is known, get room coordinates (`clickTagTruth.m`). The five anchors were positioned exactly this way — clicked, not taped (`clickAnchorsWorld.m`) — which is why the anchor file's layout is named `clicked_kinect`.

Try it yourself

To measure a parked tag's true position (used throughout calibration):

```
truth = clickTagTruth(0.24, struct('uwbTagIds', 240));
% 0.24 = the tag antenna's height above the floor in metres.
% Click the antenna in the snapshot; world (x,y) comes back.
```

The height argument matters: a click is a ray from the camera, and the height picks the point along that ray. An error of Δz shifts the answer by roughly $\rho\,\Delta z/h$ — at 2 m from the nadir, 1 cm of height error is ∼5 mm of position error.

Part III

Measuring distance with radio

5 Two-way ranging

5.1 The idea, and why it takes four timestamps

A tag cannot simply timestamp a message and have the anchor timestamp its arrival: the two clocks are not synchronised, and never will be to picoseconds. Instead the tag times a *round trip*, which uses only its own clock. But then the anchor's processing delay sits inside the measurement, and worse, the two clocks tick at very slightly different *rates* (crystal oscillators differ by parts per million — at nanosecond scales that matters).

The fix is **double-sided two-way ranging**: both sides measure both a round trip and a reply delay, and the four numbers are combined as

$$t_{\text{flight}} \;=\; \frac{T^{\text{tag}}_{\text{round}}\,T^{\text{anc}}_{\text{round}} \;-\; T^{\text{tag}}_{\text{reply}}\,T^{\text{anc}}_{\text{reply}}}{T^{\text{tag}}_{\text{round}} + T^{\text{anc}}_{\text{round}} + T^{\text{tag}}_{\text{reply}} + T^{\text{anc}}_{\text{reply}}}, \qquad d = c\,t_{\text{flight}}. \tag{2}$$

In plain words

Each side asks: "how long did I wait in total, and how long did I spend replying?" Multiplying and dividing the four durations in this particular pattern makes the unknown clock-rate difference cancel to first order. We specifically audited that our firmware uses this product form (in `TwrEngine.cpp`) — because if it had used the simpler averaging form, clock drift would have been a suspect for the drift problem of §11. It is not.

The chip's time unit sets the ruler: one DW1000 tick is $1/(128 \times 499.2\,\text{MHz}) \approx 15.65\,\text{ps}$, which at the speed of light is

$$1 \text{ tick} \;\approx\; \mathbf{4.69\ mm}. \tag{3}$$

This number appears throughout the calibration tools: one antenna-delay tick moves every measured distance by 4.69 mm.

5.2 How a measurement round actually runs

One tag ranges to the five anchors *in turn* — one exchange at a time, so radios never talk over each other (`UwbScheduler.cpp`). A full sweep of five anchors takes roughly 125 ms, giving the 5–7 Hz update rate. An anchor that fails twice in a row is skipped for a few sweeps with exponential back-off, so a dead anchor cannot stall the ring. With two tags, a token-passing ring (`TagRing.cpp`) alternates sweeps between them — which is why each tag reports at ∼3 Hz when both run.

After each sweep the tag prints one text line (and, if WiFi is up, broadcasts the same line as a UDP packet). The format is deliberately boring — versioned ASCII, one line per sweep:

```
RTLS,v4,<ms>,<tag>,<n>,{id,mm,rx,fp,q,cfo,tex}×n,DIAG,temp,vbat[,IMU,...]
```

carrying per anchor: distance (mm), received power and first-path power (dBm), quality, carrier-frequency offset and slot timing (diagnostics added during the investigation of §11); then radio temperature and battery; then the IMU sample if the tag has one. Parsed by `parseRtlsLine.m`, which accepts both v3 and v4.

Pitfall — this cost us real time

A failed diagnostic read inside the chip yields the sentinel value `-2147483648` in the power fields. Early on this poisoned averages silently. Every consumer now rejects sentinel-bearing measurements explicitly (see `SENTINEL` in `solveSweep.m`).

6 Calibrating the radios

Raw DW1000 distances are wrong by metres. Two corrections bring them to centimetres.

6.1 Antenna delay — and the trap in stored memory

The signal spends time inside each board — silicon, PCB traces, the antenna itself — before it physically leaves. The timer cannot tell this from flight time, so every board carries a calibration constant ("antenna delay") subtracted in hardware. Each tick of it is the 4.69 mm above.

Pitfall — this cost us real time

The delay constants live in the ESP32's non-volatile storage (NVS), and **NVS survives re-flashing**. Boards carried garbage values from months earlier: one anchor measured +4 m, another +1 m, consistently, with tight scatter — it looked exactly like a software bug. The tell was that the errors were *constant per anchor* and survived every host-side change. A factory reset of the stored values fixed it instantly. Any future bring-up should wipe NVS first. (The boot banner now always prints which delay source is active, so this is visible at every connect.)

Calibration then proceeded against camera truth: park the tag at a clicked reference point, compare each anchor's median measured distance with the true distance, convert the error to ticks (÷4.69 mm), push the adjustment to the anchor over the air (`SETANTDELAY`), repeat. Implemented in `tune_delays_sweep.m`; the tag's own delay is tuned the same way by `tune_tag_delay.m` (tag 241 converged in a single step: a −507 mm common offset became −2 mm).

Measured result

After two rounds, all five anchors measured within ±16 mm at the reference point, and the solved position there landed **4 mm** from camera truth.

6.2 An anomaly we could not explain

During calibration we found that the *same* ranging routine gives a different answer depending on how it is called. Rapid back-to-back exchanges (the burst mode used by a hardware self-calibration command) and the normal round-robin sweep disagree by a constant per-anchor offset between +60 and +500 mm — reproducible across runs, linear at exactly 4.72 mm per delay tick within each mode. Same code, same boards, different calling pattern.

We never found the mechanism. Later analysis (§11.1) showed a small real timing-state effect exists (±5–7 mm), far too small to explain this. The pragmatic fix: **calibrate on the exact path production uses**, so whatever the effect is, it cancels. Recorded honestly as open.

6.3 Strong signals measure long: the power bias

With delays calibrated, a clean pattern remained in the data: distance error correlated with received signal strength at $\rho = +0.78$. Strong signal, reads long; weak signal, reads short. This is a documented DW1000 characteristic (the leading-edge detector fires slightly differently with amplitude), and the vendor's published coarse correction is already applied inside the driver — what we see is the residual on top.

We fitted, per anchor k, a straight line in the *first-path* power p_{fp} (the strength of the earliest signal component — more stable than total power):

$$d_{\mathrm{corrected}} \;=\; d_{\mathrm{measured}} \;-\; (a_k\, p_{\mathrm{fp}} + b_k), \qquad a_k \in [7.7,\, 9.5]\ \mathrm{mm/dB}. \tag{4}$$

The fit lives in `range_correction.json`, is loaded by `loadRangeCorrection.m`, and is applied inside `solveSweep` with the power clamped to the fitted range so the line is never extrapolated. Validated leave-one-position-out: fitting on 18 of the 19 campaign positions and testing on the held-out one.

Measured result

Floor-wide single-reading accuracy: **199 mm → 94 mm RMSE**. This one correction did more than any other single change in the project.

We also tried a neural network and a physics-informed network for this correction. Both lost to the straight line — see §16.3.

7 Getting the data to MATLAB

Two interchangeable transports feed the host, with the same interface (`start` / `drain` / `drainEvents` / send):

- `TagSerial.m` — USB serial, 115200 baud. Opening the port resets the tag (DTR line), which is actually useful: you always see the boot banner, including which antenna delay is active.
- `TagUdp.m` — WiFi. The firmware broadcasts every sweep to port 4100 already; this class just listens. One socket receives *both* tags and the consumer separates them by the tag id in each line. It also learns each tag's IP from its traffic, so commands can be sent back.

Pitfall — this cost us real time

Two traps here. First, MATLAB's own `udpport` needs a toolbox we do not have — so `TagUdp` uses the Java `DatagramSocket` that ships inside MATLAB. Second, an *empty* Java socket poll costs ∼15 ms (it works by timeout exception). Harmless in a display loop; catastrophic inside a 20 Hz control loop, as §16.11 recounts. Poll once per loop, not twice.
Also: the first bind pops a Windows Firewall prompt. Decline it and packets vanish silently forever.

Try it yourself

Is the network delivering at all? This prints every arriving line and a per-tag summary, and is the first thing to run when WiFi "does not work":

```
udp_probe(15)        % listen 15 s on port 4100
```

For years the project believed the campus WiFi blocked UDP. `udp_probe` disproved that in fifteen seconds. Measure; do not inherit beliefs.

Part IV

From distances to positions

8 The least-squares position solver

8.1 The cost function

Given corrected distances d_k to anchors at known positions $\mathbf{a}_k$, the solver (`multilaterate.m`) finds the tag position $\mathbf{p} = (x, y)$ minimising

$$J(\mathbf{p}) \;=\; \sum_k w_k \Big(\underbrace{\|\mathbf{p} - \mathbf{a}_k\|}_{\text{distance if the tag were at } \mathbf{p}} \;-\; \underbrace{d_k}_{\text{distance measured}} \Big)^2 . \tag{5}$$

The tag's height is fixed and known (24 cm — it rides a plate, it does not fly), so only x and y are solved. That makes the problem small, fast, and hard to break.

In plain words

For a candidate position, ask each anchor: "if the tag were here, how wrong would your measurement be?" Square those wrongnesses (so sign does not matter and big errors hurt disproportionately), weight them by how much you trust each anchor, add them up. The position with the smallest total wins.

Because distance is a curved function of position there is no closed-form answer. Levenberg–Marquardt iterates from a starting guess: each step solves

$$\boldsymbol{\delta} = -\big(J^\top W J + \lambda \,\mathrm{diag}(J^\top W J)\big)^{-1} J^\top W\, \mathbf{r}, \tag{6}$$

where $\mathbf{r}$ is the vector of per-anchor errors and J its sensitivity to moving $\mathbf{p}$. The damping λ makes it cautious when a step fails and bold when steps succeed. Warm-starting from the previous position means it typically converges in two or three iterations.

8.2 Three defences that matter

Outlier gate. After a first solve, the spread of the errors is estimated robustly with the median absolute deviation, $\hat{\sigma} = 1.4826 \cdot \mathrm{median}|r_k - \tilde{r}|$ — a spread estimate that one wild value cannot inflate. Any measurement further than $3\hat{\sigma}$ out is dropped and the solve repeated. A floor of 2 cm stops the gate from becoming paranoid when all residuals happen to be tiny.

Collinearity guard. If the anchors that answered lie nearly on a straight line, the two mirror-image positions across that line fit *equally well* — the data genuinely cannot decide. The solver checks the spread of the answering anchors perpendicular to their principal line (the second singular value of their centred coordinates) and **refuses to answer** below 0.3 m. This is not cosmetic: when two anchors died mid-run and left three nearly-collinear ones, this guard was the only honest response (§16.6).

Signal-quality weights — covered in full in §8.3 below, because the story of what the NLOS machinery does (and honestly does not) earn deserves more than a paragraph.

The full per-sweep pipeline — sentinel rejection, power correction, weights, solve, diagnostics — is assembled in `solveSweep.m`, which is the *single* solve path used by every live tool and every replay. One code path means offline results and live results can never quietly diverge.

8.3 Blocked paths: the NLOS machinery in detail

8.3.1 The physical signature

When the straight line from tag to anchor is clear (*line of sight*, LOS), most of the received energy arrives in the first instant — the direct ray. When something blocks that line (*non-line-of-sight*, NLOS — a person, the rover body, furniture), the direct ray is attenuated but reflections off walls and floor still arrive, *later*, because their paths are longer. The chip reports both the total received power p_{rx} and the power in the earliest-arriving component p_{fp}, so the situation is visible in one number:

$$g \;=\; p_{\mathrm{rx}} - p_{\mathrm{fp}} \quad [\mathrm{dB}]. \tag{7}$$

In plain words

Think of a shout across a courtyard. In the open, the loudest thing you hear is the direct shout, and the echoes are quiet afterthoughts: small gap. If a wall blocks the shouter, all you hear *is* echoes — lots of total sound, none of it early: large gap. The gap is a blocked-path detector that needs no extra hardware, and it is the vendor's own recommended metric (application note APS006). Crucially, an NLOS distance is not just noisier — it is *biased long*, because the energy that does arrive travelled a longer path. That is why a blocked anchor should count less even before it looks like an outlier.

One same-frame subtlety in the firmware is worth recording: both powers must be read from the *same received frame*. The anchor's report carries a power measured on the *other* leg of the exchange; comparing that against the tag's own first-path reading would make the gap physically meaningless. The tag therefore overwrites the reported value with its own same-frame measurement (see the diagnostics block in `TwrEngine.cpp`).

8.3.2 From gap to weight

`gapWeights.m` converts the gap into a per-anchor trust weight:

$$w(g) \;=\; \max\!\Big(0.05,\; 10^{-\max(0,\,g-3)/10}\Big), \qquad w(\text{no diagnostics}) = 1. \tag{8}$$

In words: up to a 3 dB gap the anchor is fully trusted (w=1; small gaps are normal even in LOS). Beyond that, trust falls by a factor of ten for every 10 dB of excess — mirroring that the gap itself is a logarithmic quantity. The floor of 0.05 means even a badly obstructed anchor is *de-emphasised, never silently discarded*: keeping a whisper of it preserves solve geometry, and the MAD gate (§8) still exists for outright rejection. A missing diagnostic (sentinel, §5.2) maps to full weight rather than a fabricated suspicion.

8.3.3 The three places the weight acts

1. **In the least-squares solve** — as the w_k of Eq. (5), inside `solveSweep.m`. A blocked anchor bends the position less.
2. **In the estimators** — the same weights ride along as per-range measurement trust: the Kalman filter's tightly-coupled range updates divide the measurement variance by w_k, and both moving-horizon estimators take them as w_{kj} in Eq. (10), so a suspect range pulls the window more gently.
3. **In the position-fix covariance** — the loosely-coupled Kalman update inflates its measurement noise R by (among other factors) the mean gap of the sweep (the `adaptiveR` helper in `live_tag.m` and `ekf_replay.m`): a sweep that was obstructed overall is trusted less as a whole.

8.3.4 What the ablation honestly showed

Good machinery still has to survive being switched off. On 19 parked dwells we ran the weights off against on: scatter was equal, slow wander was equal, and static accuracy was actually *slightly better with the weights off* (48 mm → 39 mm median). We kept them on anyway — a deliberate judgement, recorded here so it can be revisited: a parked-room test never exercises the case the weights exist for, which is *transient* blocking — a person stepping between tag and anchor, the rover's own body shadowing a link mid-turn. In those moments the biased-long range arrives with a large gap attached, and down-weighting it is exactly right. The static cost of carrying the weights is a few millimetres; the protection is against decimetre-scale transients.

Two relatives of the idea were tried and dropped. A *hard* drop of high-gap ranges added nothing over soft weighting in the correction bake-off (§6.3) — unsurprising in hindsight, since the floor-weighted soft version already contains it. And the chip's separate "receive quality" metric turned out to duplicate what power already says (correlation 0.52 with error, essentially nothing beyond it) and was crossed off.

8.3.5 And the CIR? Diagnosis only — so far

The channel impulse response of §11 is the *full* version of the story the gap summarises in one number: energy per ≈30 cm of path, tap by tap. It would be natural to ask whether it feeds the pipeline. It does not — its one use was diagnostic, and it was decisive there (it settled the parked drift's root cause). Two designed-but-deferred uses sit in the plan: streaming a short CIR tail with every exchange to fit a *per-reading error model* (predict each range's error from the echo shape, rather than from power alone), and revisiting the learned power correction of §16.3 with CIR features once dense moving truth exists. The CIR-as-input idea is arguably the most promising unexploited item in the backlog.

9 The static baseline, and an important ceiling

To measure honestly, we parked the tag at **19 camera-clicked positions** across the floor (∼20 s each, ∼7 600 readings total) using `static_accuracy.m`.

Table 4: Single-reading 2-D error over all 19 positions (no time filtering).

	RMSE	Median	95th pct.
Delay calibration only	199 mm	132 mm	393 mm
+ power-bias correction	**94 mm**	**69 mm**	160 mm
Oracle: a perfect per-spot constant	72 mm	31 mm	—

The *oracle* row is the most informative experiment in the project. We asked: if we cheated and subtracted each position's own average error — the best any *fixed* correction could ever do — what remains? Answer: 72 mm, not zero.

In plain words

That means the leftover error is not a map of the room waiting to be measured. It *changes over time while the tag sits still.* No lookup table, no smarter fixed calibration, no denser survey can remove it. Something in the radio environment is slowly moving. Chasing that something is §11.

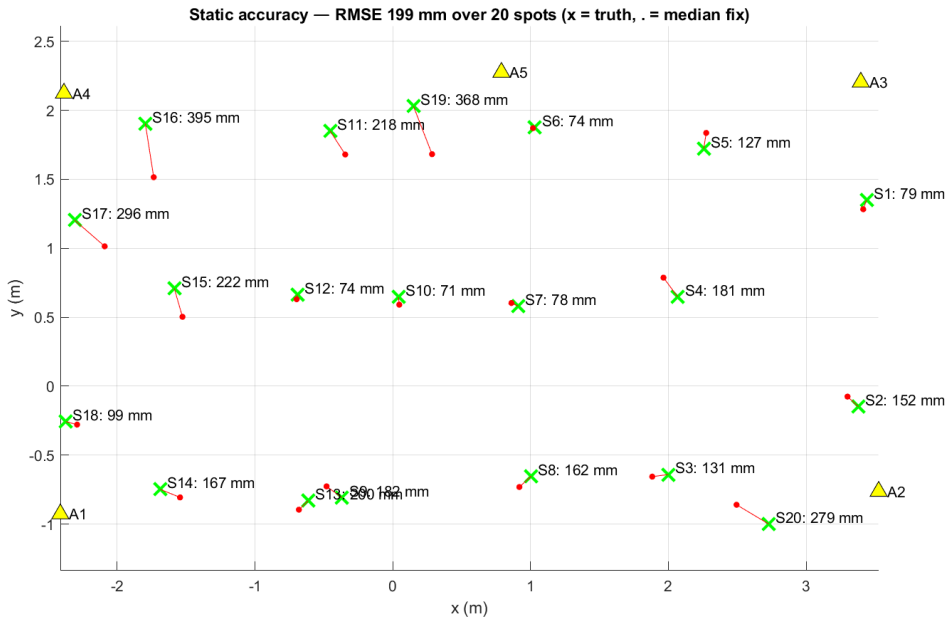

Figure 1: The static campaign: truth positions (green ×), median solved positions (red dots), and per-spot error. The variation across the floor is the spatial error field that the power correction then largely removes.

10 The Kalman filter

A single reading is noisy; the rover moves smoothly. A Kalman filter (`FusionEkf.m`) exploits that by carrying a *belief* — position, velocity, and how uncertain both are — forward through time, correcting it with each new fix.

Predict. Between readings the state $\mathbf{x} = [p_x, p_y, v_x, v_y]^\top$ advances under constant velocity,

$$\mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\,\Delta t, \qquad \mathbf{v} \leftarrow \mathbf{v} + (\text{unknown acceleration}), \tag{9}$$

and the uncertainty grows by how much acceleration we are willing to imagine ($\sigma_a = 0.8\,\mathrm{m/s^2}$ while moving). This knob is the responsiveness/smoothness trade: too big and the filter chases noise, too small and it lags corners.

Update. A new fix $\mathbf{z}$ is compared with the prediction; the surprise $\mathbf{y} = \mathbf{z} - \hat{\mathbf{p}}$ is weighed against the combined uncertainty S, and the state moves part-way toward the measurement: $\mathbf{x} \leftarrow \mathbf{x} + K\mathbf{y}$ with gain $K = PH^\top S^{-1}$. The scalar $\mathbf{y}^\top S^{-1}\mathbf{y}$ ("how many sigmas of surprise") is χ^2-gated at 95 %.

Five additions, each earned by an observed failure:

1. **Robust updates, not hard rejection.** A fix slightly beyond the gate is not discarded; its assumed noise is inflated so its surprise sits exactly at the gate — it still pulls, just gently (the Huber idea, borrowed from the Bitcraze flight stack). Only fixes beyond 10× the gate are rejected outright. One failing mode improved 164 mm → 105 mm from this change alone.

2. **ZUPT.** When the IMU's recent window is quiet ($\max|\mathbf{a}| < 0.12\,\mathrm{m/s^2}$, $\max|\boldsymbol{\omega}| < 0.05\,\mathrm{rad/s}$ over 8 samples), feed a pseudo-measurement $\mathbf{v} = 0$. The window matters: instantaneous thresholds fire mid-stride.

3. **stillMode.** ZUPT alone did not pin a parked tag — the process noise kept re-opening the gain (∼30 mm of fresh position uncertainty per step). When still, σ_a drops to $0.05\,\mathrm{m/s^2}$ and the dot genuinely settles.

4. **Per-anchor bias states.** Each anchor's residual bias is learned as an extra state, $d_k = \|\mathbf{p} - \mathbf{a}_k\| + b_k$, a slow random walk that is *remembered while the anchor is absent*. Why: when the answering subset changes, the average bias changes, and the position used to jump 13–30 mm. With bias memory: **6–9 mm**.

5. **RangeHold** (`RangeHold.m`). The complementary trick at the measurement level: an anchor that misses a sweep contributes its recent median for a short grace period, so the solve geometry never collapses. Subset jumps at the raw level: 13–28 mm → **6–10 mm**.

Try it yourself

Everything above can be re-run offline against any logged session:

```
ekf_replay("matlab/logs/serial_raw_20260721_202505.log")
% with a static campaign for scoring against truth:
ekf_replay(srcLog, spotsJson="matlab/results/static_accuracy_.../spots.json")
```

11 The slow "breathing", and how we hunted it

With all of the above, a parked tag still wandered **5–7 cm over seconds to minutes** — slow, smooth, and indistinguishable from genuine creeping motion. This section matters beyond this project: it is a worked example of isolating an error you cannot remove.

11.1 Phase A: interrogating existing logs

Three hypotheses were tested purely on recorded data (`phase_a_forensics.m`, findings in `phase_a_findings.md`)

A1 — is it measurement timing? The anomaly of §6.2 suggested distance depends on exchange cadence, and production cadence wanders (retries, skips). Across 44 000 measurements, the correlation between each anchor's realised timing gap and its distance error was $|\rho| \leq 0.06$. *Rejected* — though a genuine micro-effect exists: a measurement following extra receiver idle time shifts by ±5–7 mm, and the first reading after a long gap reads +39 mm. Two orders of magnitude too small.

A2 — is it the tag or the links? The decisive split. If the tag's clock/temperature drifted, *all five* distances would breathe together (common mode). If the radio paths are each doing their own thing, they breathe independently. Measured cross-anchor correlation of the slow residuals over 23 parked dwells: **+0.05** — independent. One number eliminated the tag's clock, temperature and battery, and pointed at the individual links.

A3 — can the filter absorb it? We tried inverting the noise budget while parked: trust position completely, let the per-anchor bias states soak up the drift. It made things dramatically worse (91 mm → 319 mm) — see §16.4 for the instructive reason.

11.2 Phase B: looking at the radio waveform itself

The DW1000 can dump its channel impulse response (CIR) — the received signal amplitude tap by tap, one tap ≈ 30 cm of path. We added a capture command to the firmware (`CIR,<anchor>` in `TwrEngine.cpp`) and a waterfall tool (`cir_capture.m`).

The picture was unambiguous. The direct path — the one whose arrival time *is* the distance — sits roughly **20 dB below the strongest reflection** and only 8–10 dB above the noise floor. With antennas 24 cm off the floor, the direct ray grazes the ground and its floor reflection arrives almost on top of it; the detector locks onto a blend, and as the blend shifts, centimetres of apparent distance come and go. A quiet empty room breathed exactly as much as one with a person walking (132 mm peak-to-peak over 81 s either way): this is the geometry, not passers-by.

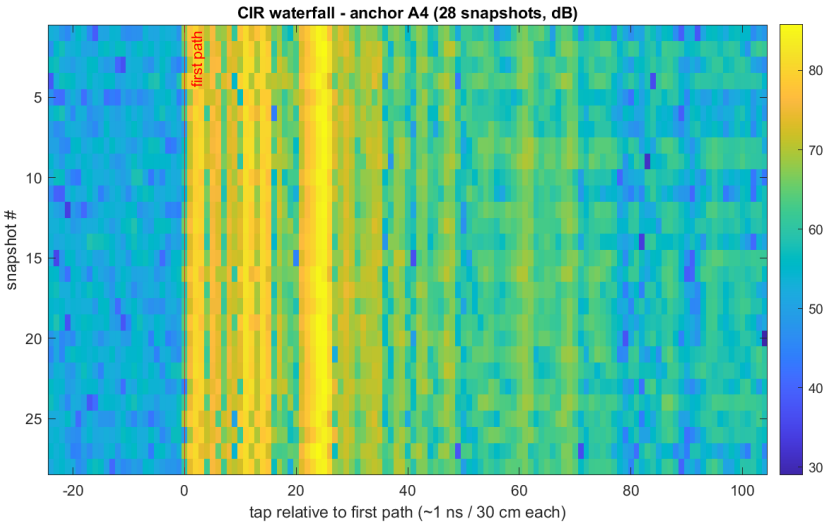

Figure 2: Twenty-eight consecutive channel snapshots to anchor A4, aligned on the detected first path (red line). The energy *at* the first path is faint; the bright band arriving ∼25 taps (≈7.5 m of extra path) later is a reflection much stronger than the direct signal.

Measured result

The breathing is per-link multipath caused by low antenna mounting. The genuine fix is physical — raise the antennas (or the tags). The anchors are permanently installed, so this is deferred, and the system is honest about carrying a 5–7 cm slow parked drift.

11.3 Living with it: the output pin and smoother

A parked rover *is* stationary — we know it from the IMU and from the solver itself. So at the **display/logging layer only**, when the system is confident it is parked, the reported pose freezes: movement within a 5 cm deadband is ignored, a sustained excursion (∼1.5 s beyond the band) re-pins, and any real motion releases instantly. While moving, a light exponential smoother ($\alpha = 0.6$) takes the high-frequency jitter off, chosen from measurement: it removed ∼50 % of jerk for 25 mm of lag, whereas tuning the filter's process noise managed only 13 % for triple the lag (§16.7).

This is deliberately cosmetic. The estimator state is untouched (A3 showed why touching it goes wrong), the raw estimate is what gets logged, and the title bar says `PIN` whenever the freeze is active. Implementation: the pin/smoother block in `live_tag.m`.

12 The moving-horizon estimator

12.1 Why look at more than one reading

A Kalman filter digests one reading and moves on; it can never revisit a decision. A **moving-horizon estimator** (MHE) keeps the last N readings (here N=10, about two seconds) and, at every step, re-solves the *entire recent trajectory*. New evidence gets to reinterpret the recent past. The cost, solved over all window states $\{\mathbf{x}_k\}$ at once:

$$\min_{\{\mathbf{x}_k\}} \underbrace{\sum_k \sum_j w_{kj}\, \rho(\|\mathbf{p}_k - \mathbf{a}_j\| - d_{kj})}_{\text{fit every distance in the window}} + \underbrace{\sum_k \|\mathbf{x}_{k+1} - f(\mathbf{x}_k)\|^2_{Q^{-1}}}_{\text{move like a real vehicle}} + \underbrace{\|\mathbf{x}_1 - \bar{\mathbf{x}}\|^2_{P^{-1}}}_{\text{respect what came before}} \tag{10}$$

The robust loss is the pseudo-Huber function

$$\rho(r) = \delta^2\Big(\sqrt{1 + (r/\delta)^2} - 1\Big), \qquad \delta = 0.15 \text{ m}, \tag{11}$$

which behaves like $r^2/2$ for small errors but grows only *linearly* for large ones — a wild measurement bends the answer instead of dragging it. The "arrival cost" (third term) is how information older than the window survives: the window's first state is softly tied to what the previous solve believed.

Implemented in `MheEstimator.m` using CasADi 3.7.0 (symbolic derivatives) and IPOPT (the solver). Median solve time: **3 ms** — easily real-time at 6 Hz.

Measured result

Same recorded data, MHE vs Kalman: while *moving*, the MHE track is **40–44 % smoother** (jerk 0.36 vs 0.61) for 6 mm extra lag. Parked, a tie (61 vs 64 mm RMSE). It is offered as an opt-in (`live_tag(..., estimator="mhe")`), not the default, because at 2 of 19 parked spots the breathing slowly drags the whole window ∼0.23 m — the average stays right, but the drift is real.

12.2 The jump guard

IPOPT occasionally lands on a different-but-similar optimum between consecutive solves, making the output leap. The guard: if the new answer jumps further from the constant-velocity prediction than 0.20 m (or the solver reports non-convergence), discard it, fall back to the *measurement-only* fix, and re-seed the window. The first version of this guard fell back to dead reckoning instead and diverged nine metres — the full story is §16.5, and the design rule it taught is printed there in bold.

12.3 Motion models: using the IMU without trusting it

We probed what the tag's IMU can honestly provide (log-based, in §16.1): its accelerometer bias is ≈0, its *turn rate* is trustworthy, but its absolute compass direction is rotated by an unknown amount. The safe pattern: use **changes**, never absolutes.

- **cv** — constant velocity, gyro optional: the velocity vector is rotated by the measured yaw rate $\omega\,\Delta t$ each step. With $\omega = 0$ it reduces exactly to plain constant velocity, so it can never be worse.
- **unicycle** — for a car-like vehicle. State $[p_x, p_y, \psi, s]$: heading and forward speed. The gyro drives the heading ($\psi_{k+1} = \psi_k + \omega\,\Delta t$, softly), and velocity is *defined* as $s\,[\cos\psi, \sin\psi]$ — sideways motion is not representable, because a front-steering car cannot slide sideways. The UWB measurements anchor the absolute heading that the gyro alone cannot know.

Try it yourself

Compare estimators on any log, no hardware needed:

```
mhe_replay("matlab/logs/serial_raw_20260721_202505.log")          % cv
mhe_replay(log, useImu=true, model="unicycle")                    % car model
motion_check("matlab/logs/rtls_log_20260721_202505.jsonl")        % metrics
```

Part V

The rover: two tags, one pose

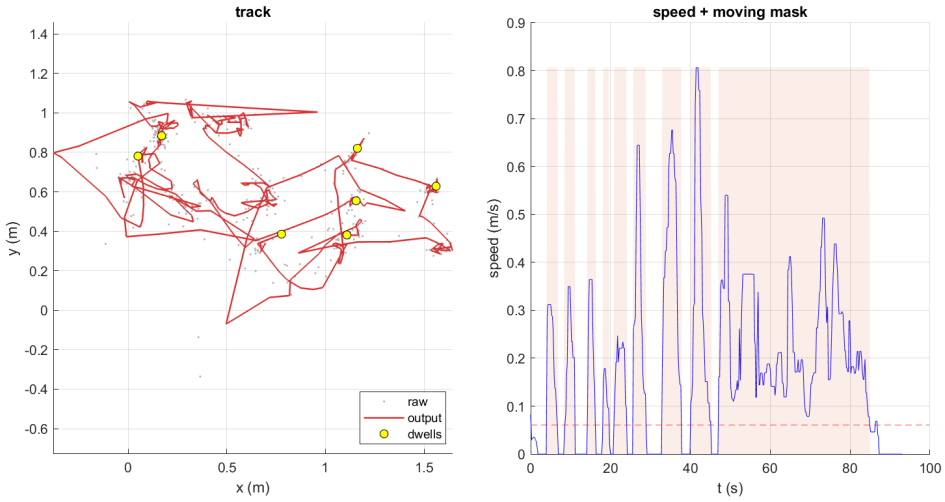

Figure 3: The hand-motion check that tuned the smoothing: raw fixes (grey), output track (red), detected dwells (yellow). Between-dwell displacements of the ∼50 cm hand moves read 0.44–0.58 m — the poor man's truth before the rover runs existed.

13 The rigid-body estimator

13.1 One body, not two dots

The two tags are bolted to one plate, $L = 0.527\,\mathrm{m}$ apart (measured antenna centre to antenna centre — it matters, this number becomes exact in the math). Estimating them separately throws that knowledge away. Instead, `MheRigid.m` estimates a single body state $[c_x, c_y, \psi, s]$ — centre, heading, speed — and *derives* both tags from it:

$$\mathbf{p}_{\text{front}} = \mathbf{c} + \tfrac{L}{2}\begin{bmatrix}\cos\psi\\ \sin\psi\end{bmatrix}, \qquad \mathbf{p}_{\text{rear}} = \mathbf{c} - \tfrac{L}{2}\begin{bmatrix}\cos\psi\\ \sin\psi\end{bmatrix}. \tag{12}$$

In plain words

This is constraint by *construction*, not by penalty — and the distinction is the whole design. A penalty says "please keep the tags 52.7 cm apart" and can be outvoted by noisy data. This parameterisation *cannot express* any other separation: there is no combination of c, ψ, s that puts the tags at the wrong distance. Three facts come for free: the separation is exact in every answer; "both stopped or both moving" is automatic (one speed state); and the heading is observed by the radios (which end is where) *and* steered by the gyro — better than either source alone.

The two tags report at different instants (the token ring alternates), so there is no pairing of readings in time. Each arriving sweep — from either tag — becomes its own node in the horizon, labelled $+1$ (front) or -1 (rear), observing the body at its own timestamp. Terrain is handled with the IMU's roll and pitch taken *directly* (they are gravity-referenced and excellent, §15): on a pitched slope the horizontal tag separation shrinks to $L\cos\theta$ and the two antennas sit at heights $z_0 \pm \frac{L}{2}\sin\theta$ — both effects are in the measurement model.

How good can the heading be? With per-distance noise σ_d and baseline L, the geometric limit per reading is roughly

$$\sigma_\psi \approx \frac{\sqrt{2}\,\sigma_d}{L} \xrightarrow{\sigma_d=30\,\mathrm{mm},\ L=0.527\,\mathrm{m}} \approx 4.6^\circ \text{ per reading}, \tag{13}$$

which filtering and the gyro then improve. A longer baseline buys heading accuracy linearly — worth remembering if the plate is ever redesigned.

13.2 Validating before trusting

A sign error in that parameterisation would be invisible live and poison everything. So before any hardware run, a synthetic test (`test_mhe_rigid.m`) drives a simulated rover on an arc over the *real* anchor layout with realistic 30 mm range noise, where the true answer is known exactly:

Table 5: Rigid estimator against known synthetic truth.

Condition	Centre (median)	Heading	Separation
Flat	14.1 mm	1.18°	0.5000 m (exact)
+8.6° pitch	14.1 mm	1.17°	$= L\cos\theta$ ✓
−11.5° pitch	14.2 mm	1.18°	$= L\cos\theta$ ✓

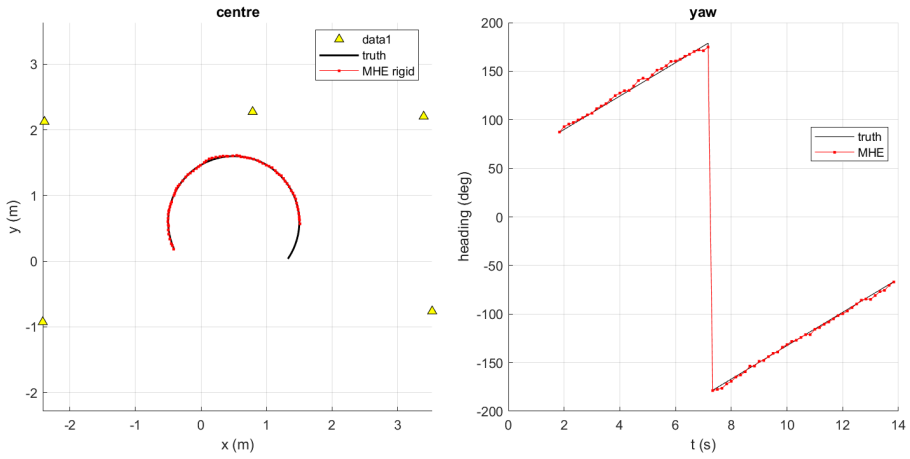

Figure 4: The synthetic validation: true arc (black) vs the rigid estimate (red), and true vs estimated heading. Zero guard trips.

Try it yourself

Live, with both tags on and broadcasting:

```
live_rover(baseline=0.527)
```

Blue and red dots are the two tags (always exactly the baseline apart — by construction), the purple star is the centre, the arrow is the heading. The title shows MHE yaw next to the raw two-fix yaw as a live consistency check. If both tags go silent, the display **freezes and says COASTING** rather than gliding away on a prediction — the lesson of §16.6 baked in.

14 Driving the rover: truth and radio on one clock

The final piece closes the loop with ground truth *in motion*. The rover already carries an AprilTag that the calibrated Kinect tracks; driving it with both UWB tags aboard records, simultaneously and on one clock: where it truly went, and what the radios said.

`rover_teleop_uwb.m` was ported from the existing MMS rover teleoperation script (used read-only — the MMS codebase is not modified). It drives the rover with a PS4 controller at 20 Hz while logging: commanded velocities, AprilTag pose, the rover's own IMU and wheel encoders, and every UWB sweep from both tags, all POSIX-timestamped.

One safety decision is worth its own paragraph: **the estimator does not run inside the driving loop.** That loop services the emergency stop; the MHE occasionally takes 90 ms; delaying an

Table 6: PS4 controls (unchanged from the original teleop).

D-pad up/down	linear speed ±0.01 m/s (max 0.10)	
D-pad left/right	turn rate ±0.01 rad/s (max 0.107)	
R1	toggle reverse	Square: straighten (turn rate 0)
Circle	**emergency stop**	Triangle: reset both to zero
PS button	finish and save	

E-stop to compute a position is a bad trade in every universe. The loop only *records* (microseconds); estimation happens offline on the recording and is bit-identical, since the sweeps are all the estimator consumes.

14.1 Scoring a run

`rover_run_report.m` turns a run into numbers. Two frame subtleties are handled explicitly rather than assumed away:

The lever arm. The AprilTag is not at the rig centre — it sits 11.35 cm behind and 10 cm to the right of it (measured). That offset *rotates with the rover*, so truth is de-rotated per sample: $\mathbf{c}_{\text{true}} = \mathbf{p}_{\text{tag}} - R(\psi)\,\boldsymbol{\ell}$, with $\boldsymbol{\ell} = [-0.1135, -0.10]$ m.

The frame fit. The camera world and the UWB anchor world are two different coordinate systems. The best rigid mapping (rotation + translation, no scale) between the two trajectories is found by the classical SVD construction: centre both point sets, form the cross-covariance $H = \sum_i \tilde{\mathbf{u}}_i \tilde{\mathbf{t}}_i^{\top}$, decompose $H = U\Sigma V^{\top}$, and take $R = V\,\mathrm{diag}(1, \det(VU^{\top}))\,U^{\top}$. Fitting this *from data* (rather than assuming the frames coincide) also acts as a cross-check: the rotation it finds should be stable run to run.

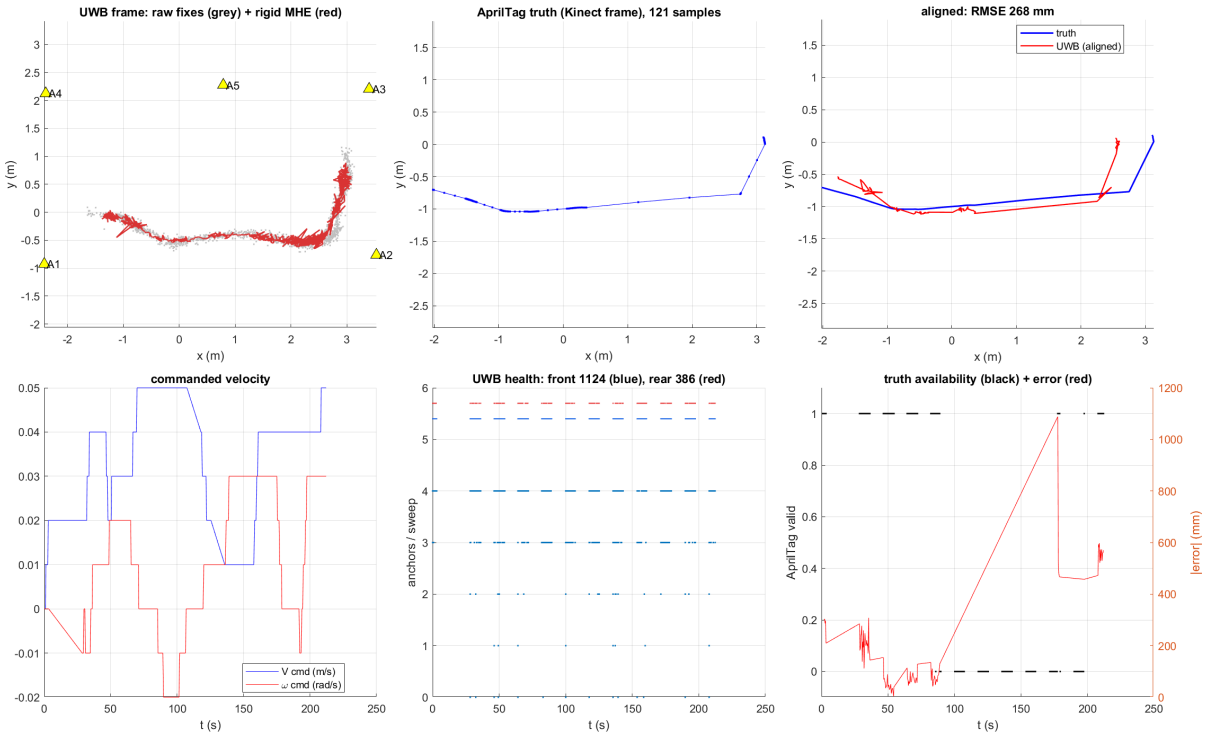

Figure 5: First driven run (212 s). Top-left: raw fixes and the rigid-MHE track in the UWB frame. Top-middle: AprilTag truth. Top-right: the two overlaid after the fitted frame transform — the shapes agree. Bottom row: commanded velocities, UWB per-sweep health, truth availability with error over time.

Measured result

First run: median error **98 mm**, RMSE 268 mm, frame fit $-3.6°$ / $[-0.46, -0.59]$ m. The RMSE is stated but **not yet trusted**, and Figure 5 shows exactly why: the camera saw the marker only 58 % of the time with one long gap (the biggest "errors" are unmeasured stretches), the rear tag was starved (386 vs 1124 sweeps — weakly constrained heading, 46 % guard trips), and only 3.6 of 5 anchors answered on average. These are setup problems, not algorithm problems, and they are the current work.

15 The full six degrees of freedom

The rover state we ultimately want is x, y, z and roll, pitch, yaw. `rover_pose_report.m` compares every channel we can observe, from every sensor that observes it (Figure 6).

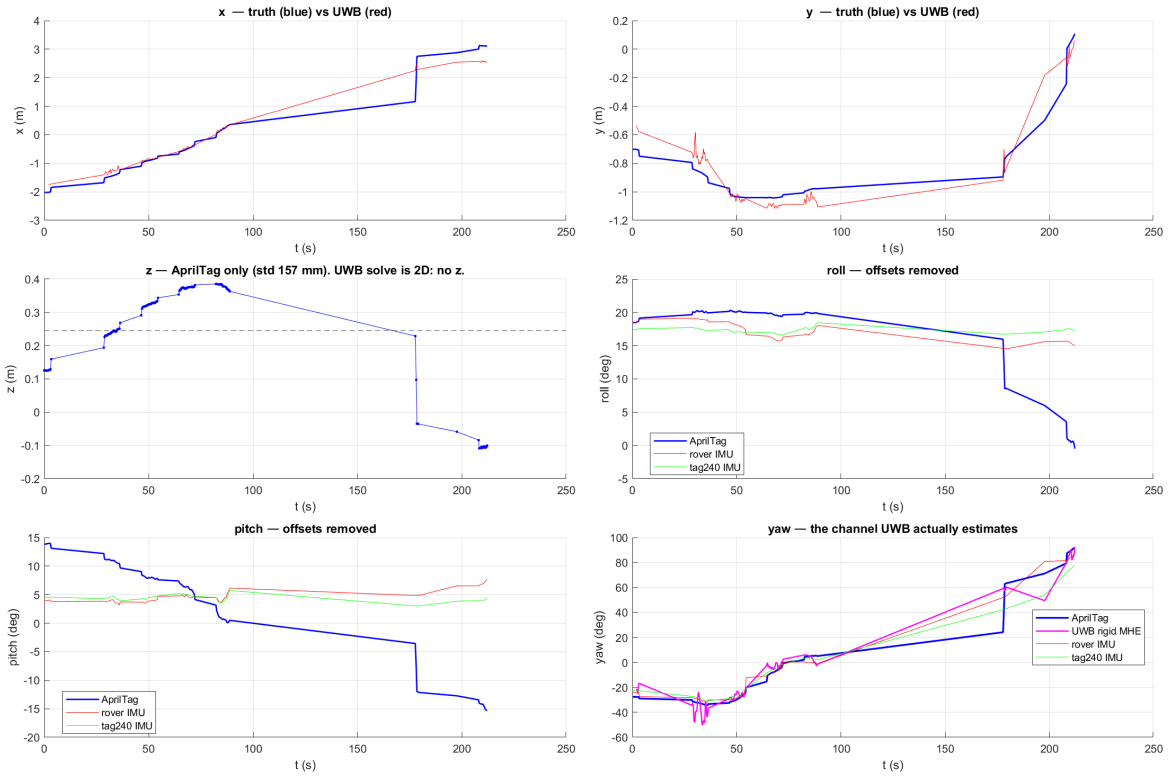

Figure 6: All six channels from the first run. Blue is the AprilTag; red/green are the two IMUs; magenta is the UWB rigid-MHE yaw. Note pitch and roll: the camera (blue) swings tens of degrees while the two independent IMUs agree with each other and stay flat — the camera is the outlier there.

Pitfall — this cost us real time

Two traps found while making this plot. The AprilTag angles are in *radians* but the rover's onboard IMU reports *degrees* — verified from the data ranges, not assumed. And every sensor defines "zero heading" differently (marker print orientation, IMU mounting, anchor frame), so channels are compared after removing a constant offset (a circular mean); the offsets themselves are reported and should stay stable run to run.

The comparison produced the most useful architectural fact of the rover phase: **the camera is only trustworthy in-plane.** Its z wanders with 157 mm standard deviation over a flat floor; its pitch and roll swing by tens of degrees while the two *independent* gravity-referenced IMUs agree with each other to a few degrees. A small, flat, distant marker simply does not constrain

out-of-plane pose well — a known weakness, now measured in our own data. The IMUs win those channels: gravity is an absolute reference.

Table 7: Who to believe, per channel — the sensors are complementary.

Channel	Best source	Quality	Truth for scoring
x, y	UWB rigid MHE	98 mm median (first run)	AprilTag (in-plane: valid)
yaw	UWB rigid MHE	**7.3° RMS**	AprilTag (in-plane: valid)
roll, pitch	IMU	few degrees, absolute	IMUs themselves (gravity)
z	— (fixed)	—	none good yet

Two notes on that table. The UWB yaw agreeing with camera yaw at 7.3° RMS — with a residual frame offset of only 1.4° after a transform fitted from *position* data alone — is a strong consistency check: heading and position tell the same story. And z is not a software gap: all five anchors sit at the same height, so the radio geometry contains almost no vertical information. If z ever matters, anchors must move to different heights; no algorithm can conjure observability that the geometry does not contain.

Part VI

Hindsight

16 What we tried that did not work

Kept deliberately, at full length. Several of these consumed more effort than the things that worked, and each ends with the rule it taught.

16.1 Predicting motion from the accelerometer

Feeding the IMU's acceleration into the filter's prediction diverged by 6–14 m. For a long time we blamed accelerometer bias — the classic culprit. Later measurement showed the bias is essentially zero ($[-0.007, -0.026, +0.013]\,\mathrm{m/s^2}$ at rest); the real fault is that the IMU's absolute orientation frame is *rotated* by an unknown amount, so perfectly good accelerations were applied in the wrong direction, and the gate then locked out the corrections that would have caught it. Gyro turn *rate* needs no absolute frame and works fine. **Rule: diagnose which property is broken (bias vs frame) before designing the fix — and prefer differential quantities when an absolute reference is unverified.**

16.2 Feeding raw distances into the filter

Updating the Kalman filter with each distance separately (tightly-coupled) is textbook-better: it works with fewer than three anchors. On real data it scored 2244 mm against 335 mm loosely-coupled. With per-distance gating, the filter can sit at a wrong position that keeps one or two distances happy and reject the rest forever — self-consistent and wrong. Robust updates later tamed it (105 mm) but never beat the simple version. **Rule: a partial-acceptance mechanism needs a global sanity anchor, or it will find a lie it likes.**

16.3 Machine learning for the power correction

The straight-line power correction (§6.3) was challenged with an MLP and a physics-informed network with a monotonicity prior. Scored honestly — leave-one-*position*-out — the line won:

98 mm vs 107 vs 108. There were 31 500 samples but only 19 independent positions; the effective sample size was 19, and extra capacity fitted noise. **Rule: count independent conditions, not rows; and make the baseline a straight line before reaching for a network.**

16.4 Absorbing the parked drift into the filter

While parked, why not trust position fully and let the per-anchor bias states soak up the breathing? Because the biases are *spot-specific*: values learned at one parking spot are wrong at the next, and the whole point of bias states is that they are remembered. The error compounded across dwells: 91 mm → 319 mm. **Rule: never let a long-memory state absorb a location-dependent error.**

16.5 The guard that dead-reckoned into the wall

The MHE jump guard originally fell back to the constant-velocity prediction. Once one bad solve slipped through, the prediction walked; every subsequent honest answer now looked like a "jump" relative to it, so the guard kept rejecting reality — a self-reinforcing runaway that ended nine metres off, with the guard firing on 73 % of solves. Falling back to the measurement-only fix instead ended the class of failure (3 % trips). **Rule: a fallback must be anchored to measurements; a fallback anchored to prediction can chase its own tail.**

16.6 Two independent tag estimates

The first dual-tag design ran two separate estimators and drew both. In one live session the two dots slid ∼3.5 m apart while the physical tags sat 30 cm apart. Two stacked causes: two anchors' batteries died mid-run, leaving three nearly-collinear anchors — so the solver *correctly* refused to answer (mirror ambiguity, §8); and with no fixes arriving, the display followed each filter's dead-reckoned prediction instead of freezing. The redesign went to the single rigid pose (§13) — which *cannot express* the tags drifting apart — plus a display that freezes and says `COASTING`, plus per-anchor health markers so a dying anchor is seen in seconds. **Rule: make impossible states unrepresentable, and never display a prediction as if it were a measurement.**

16.7 Tuning the motion model for smoothness

The obvious smoothness knob — filter process noise — was swept from 0.8 down to 0.1 m/s^2: only 13 % less jerk for 3× the lag. The roughness lives at the same timescale as genuine manoeuvres, so the model cannot separate them; a light *output* smoother achieved 50 % for 25 mm of lag. **Rule: identify which layer owns an error before tuning; cosmetic errors get cosmetic fixes.**

16.8 The gyro turn model that showed nothing — yet

On hand-carried test data, adding the gyro coordinated-turn made no measurable difference (jerk 0.357 vs 0.362). The data contained no sustained turns to exploit — a hand slides; it does not arc like a car. Unvalidated is not disproven; the rover's arcs are where it should pay. **Rule: a negative result only counts if the data actually exercises the mechanism.**

16.9 Correcting a lens that needed none

Fitting distortion to a lens with < 1.5 px of bow produced garbage parameters from click noise (§4). **Rule: measure the size of an effect before modelling it.**

16.10 Calibration that survived reflashing

The NVS trap of §6.1: stored calibration outliving firmware, causing metre-scale "bugs". **Rule: wipe persistent storage on bring-up, and make devices print their active calibration at**

boot.

16.11 The 20 Hz loop that became 1.4 Hz

When UWB recording was added to the rover teleop, the PS4 stopped "working". The controller was fine — the loop had silently slowed to 1.4 Hz, and a 200 ms button press fits entirely between two 700 ms polls. Three self-inflicted costs: the big recording arrays were placed *inside* the struct that a logging function takes by value (MATLAB copy-on-write duplicated ∼13 MB per iteration); the UDP socket was polled twice per loop at ∼15 ms per empty poll (Java timeout exception); and a timezone-aware timestamp call cost 0.64 ms each. All were found by measurement, not inspection (§the benchmark lives in the commit message of `2703da1`). The loop now prints its own achieved rate and warns below 8 Hz. **Rule: real-time loops get instrumented, always — a slow loop looks exactly like broken hardware.**

16.12 Deferred, not failed

Two designed-but-parked items, both aimed at the breathing's root cause: raising the tags/antennas out of the ground-grazing zone (the anchors cannot move; a mast for the tags is the practical variant), and a broadcast ranging protocol (one poll, all anchors answer in slots — all boards need reflashing). And one firmware fix sits staged but unflashed: the diagnostic temperature/CFO reads currently return zeros due to two read-timing bugs, found and fixed in commit `ac5743c`.

17 Where things stand, and the road ahead

Built and working end to end: calibrated ranging, robust solving, two filters, a rigid two-tag pose with heading, wireless transport, live displays with honest failure behaviour, a teleoperated rover that records truth and radio together, and offline scoring. The build phase is closed; the work now is **testing and tuning**:

1. **Data quality of truth runs** — keep the AprilTag in the camera's view (58 % coverage made the first run's RMSE meaningless); find why tag 240 was starved of sweeps; keep all five anchors alive (fresh batteries, watch the health markers).
2. **Score the estimator family** on clean runs: Kalman vs MHE-cv vs MHE-unicycle vs rigid — same logs, same truth, one table.
3. **Validate the gyro-turn and no-slip models** on real arcs, checking the gyro sign convention on the vehicle.
4. **If parked drift must shrink**: the physical fixes — tag mast first, then the deferred diagnostics (locked-room dwell with the repaired temperature/CFO telemetry).

The honest closing numbers: **94 mm** for one raw reading parked; **60–65 mm** filtered; **7.3°** heading against truth; **14 mm / 1.2°** for the rigid estimator when the data is clean (synthetic); and a moving-accuracy number that awaits a clean run to be worth printing.

Appendices

A Every runnable tool

B Key parameters and where they live

Table 8: Scripts in `matlab/scripts/`, grouped by purpose.

Live	
`live_tag.m`	One tag, serial or UDP; EKF or MHE; pin, smoother, anchor health.
`live_rover.m`	Both tags over UDP; rigid pose, heading arrow, coast guard.
`rover_teleop_uwb.m`	PS4 drive + truth + UWB capture on one clock.
Calibration	
`tune_delays_sweep.m`	Anchor antenna delays against clicked truth.
`tune_tag_delay.m`	A tag's own delay.
`anchor_delay_tool.m`	Read/set/reset NVS delays over the air.
`static_accuracy.m`	Multi-spot campaign harness.
Analysis / replay	
`replay_rtls.m`	Re-run the exact live chain from a log.
`ekf_replay.m`	Kalman study on a log, with static scoring.
`mhe_replay.m`	MHE vs EKF comparison.
`motion_check.m`	Still/moving segmentation, displacement and smoothness metrics.
`phase_a_forensics.m`	The breathing investigation (A1/A2).
`cir_capture.m`	Channel impulse response waterfalls.
`test_mhe_rigid.m`	Synthetic rigid-body validation.
`rover_run_report.m`	Trajectory vs truth scoring.
`rover_pose_report.m`	Full 6-DOF channel comparison.
`udp_probe.m`	Is UDP arriving at all?
`joy_probe.m`	What does the game controller actually report?

C Glossary

Anchor / tag
Fixed radio at a known position / mobile radio being located.

DS-TWR
Double-sided two-way ranging; the four-timestamp exchange of §5 that cancels clock-rate error.

Antenna delay
Per-board constant for time spent inside the electronics; 1 tick = 4.69 mm.

First-path power
Strength of the earliest-arriving signal component — the direct ray, ideally.

NLOS
Non-line-of-sight: the direct path is blocked; energy arrives late via reflections.

CIR
Channel impulse response — the received energy tap by tap (≈30 cm per tap); the radio's own picture of the room's echoes.

ZUPT
Zero-velocity update: telling the filter "it is stationary" as a measurement.

NIS / χ^2 gate
"How many sigmas of surprise" a measurement carries, and the acceptance threshold on it.

MHE
Moving-horizon estimation: re-solving the recent trajectory over a sliding window (§12).

Table 9: Defaults that shape behaviour. All changeable per-call.

Parameter	Default	Meaning / where
`tagZ`	0.24 m	Tag antenna height; fixes the solve to 2-D.
`sigmaAccelCV`	0.8 m/s^2	Filter responsiveness while moving.
`sigmaAccelStill`	0.05	. . . and while parked (stillMode).
`pinRadius`	0.05 m	Output-pin deadband (`live_tag`, `live_rover`).
`smooth`	0.6	Output smoother strength; 1 = off.
`horizon`	10–12	MHE window length (sweeps).
`huberDelta`	0.15 m	Where the robust loss switches to linear.
`maxJump`	0.20 m	MHE jump-guard threshold.
`baseline`	0.527 m	Tag separation — exact in the rigid model; re-measure if the plate changes.
`gapThreshDb`	3 dB	NLOS weight knee (rx−fp gap).

Arrival cost
The MHE term that ties the window's start to prior belief, so old information is not forgotten.

Unicycle model
Vehicle model with heading + forward speed and no sideways motion — a front-steering car.

Breathing
Our name for the slow (5–7 cm, seconds-to-minutes) parked drift caused by ground-grazing multipath (§11).

Lever arm
The fixed offset between a sensor and the reference point it is supposed to measure; rotates with the body.

References

- DecaWave, *DW1000 User Manual* — the radio's registers, timestamps, and diagnostics.
- DecaWave application notes *APS011* (received-power bias) and *APS006* (NLOS detection metrics).
- Neirynck et al., *An alternative double-sided two-way ranging method*, 2016 — the product-form formula of Eq. (2).
- Thomas Trojer's `arduino-dw1000` driver (Apache 2.0) — the vendored base of `dw1000/`.
- Andersson et al., *CasADi: a software framework for nonlinear optimization*, and Wächter & Biegler, *IPOPT* — the tools behind §12.
- Umeyama, *Least-squares estimation of transformation parameters between two point patterns*, 1991 — the frame fit of §14.
- Olson, *AprilTag: a robust and flexible visual fiducial system*, 2011.